

단일기계 스케줄링 - 작업 도착시간과 준비시간이 있는 경우

Cmax ≠ Constant

동적작업 도착시간만 존재하는 경우

Earliest Release Time (ERT) 규칙 → Cmax 최소화, 유류시간 최소화

하나의 작업이 완료되는 시점 t에 작업이 n개 있을 때, 가장 일찍 도착한 작업을 다음 작업으로 선택 → Next job = argmin_i {r_i}

Minimum Slack Time (MST) 규칙 → ΣWT, nT 최소화

작업을 할당해야 할 (이전 작업의 종료시점) t에 작업이 n개 있을 때, 다음 작업은 남지로부터 총 잔여작업시간을 뺀 여유시간이 가장 작은 작업 선택

Next job = argmin_i {s_i} (가장 빠른 도착시간 = 최소 t)
s_i = max{0, d_i - total remaining processing time - t} (slack)
→ 작업 i의 모든 후속공정의 가공시간 합.

Critical Ratio (CCR) 규칙 → ΣWT, nT 최소화

작업을 할당할 t시점에 작업이 n개 있을 때, 남기와 현재시간의 차이를 잔여작업시간으로 나눈 값인 긴급도를 비교하여, 작은 값을 가지는 작업일수록 긴급한 것으로 고려하여 먼저 선택

Next job = argmin_i {CR_i}
CR_i = (d_i - t) / total remaining processing time
(CR > 1) 여유있음. (CR = 1) on time (CR < 1) 늦은 것 (CR < 0) 이미 늦은 것

* MST에서 slack = 0인 경우 CCR은 우선순위 관련 없음

Apparent Tardiness Cost (ATC) 규칙

작업을 할당할 t시점에 작업이 n개 있을 때 ATC규칙은 우선순위를 결정하는 다음과 같은 인덱스 함수 I_i(t)를 사용

W_i, p_i, d_i : 상수
(합성)
I_i(t) = (W_i / p_i) * exp(-max{0, d_i - p_i - t} / k * p̄) * e^{-s_i} (1)

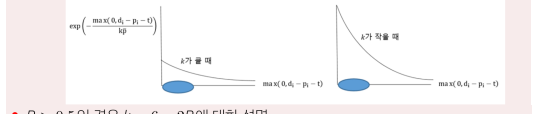
• p_i는 작업 i의 total remaining processing time을 의미
• p̄는 시간 t에 기계 앞에서 기다리는 작업들의 평균 가공시간(혹은 평균 total remaining processing time)
• 모수 k는 look-ahead parameter라고 부르며 다음과 같이 설정

두 개 이상 규칙들의 성질을 함께 지닌 규칙.
k = 4.5 + R for R ≤ 0.5
k = 6 - 2R for R > 0.5

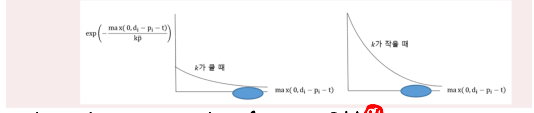
하나를 기준으로 잡고 나머지의 인덱스 조절
where R = (d_max - d_min) / C_max ≈ due date range factor
여기서 C_max는 시점 t에 대기중인 작업의 총 가공시간임
• t시점에서 가장 큰 I_i(t) 값을 갖는 작업 j가 스케줄의 맨 앞에 위치하게 됨

가공시간이 짧고 여유시간이 작은 작업일수록 우선순위 (=인덱스)가 올라감

- R ≤ 0.5인 경우 k = 4.5 + R에 대한 설명
1. R ≈ 0일 때, 대부분의 작업이 지연되지만 일부 지연되지 않은 작업의 중요도를 높이기 위해 k 값을 작게 설정 (k = 4.5) 하여 MST의 비중을 높임 (즉 MST가 잘 작동하도록 설정)
2. R = 0 → 0.5일 때, R이 증가할수록 지연되는 작업의 비율이 작아져 MST가 제 역할을 할 수 있게 k 값을 증가시킴 (k = 4.5 → 5.0)



- R > 0.5인 경우 k = 6 - 2R에 대한 설명
1. R = 0.5 → 1일 때, R이 증가할수록 여유시간이 존재하는 작업이 많아져 MST가 제 역할을 할 수 있도록 k 값을 작게 설정 (k = 5.0 → 4.0).
2. R ≥ 1이면, 여유시간이 있는 작업이 더 많아지고, 따라서 k 값을 더욱 작게 설정 (k = 4.0 이하) 하여 MST의 비중을 높임.



Modified Due-Date (MDD) 규칙

작업을 할당할 t시점에서 현재의 시간에 잔여작업시간을 더한 값과 남기를 비교하여 큰 값을 새로운 남기로 수정하여 남기값이 빠른 작업부터 선택

Next job = argmin_i {d_i^new}, d_i^new = max{d_i, t + total remaining proc. time}

* 이미 늦은 작업의 due date를 늦추는 점에서 EDD보다 좋음.

Cost Over Time (Covert) 규칙

- 1. Priority 계산: ① d_i ≤ t + p_i → Pr_i = 1 ② d_i ≥ (t + Σp_j) → Pr_i = 0
③ t + p_i < d_i < (t + Σp_j) → Pr_i = [(t + Σp_j) - d_i] / [Σp_j - p_i] = 최대 지점시간 / 최대 대기시간
- 2. 단위시간당 비용 CF_i 계산: CF_i = (Priority_i * C_i) / p_i = 단위시간당 비용 / 단위시간당 지점비용
→ argmax_i {CF_i}

Beam Search BNB와 달리 하한형계공식 사용 X. 최적해 보장 X 규칙이 추가되어야

매 노드마다 실행가능해를 만들어야 함 (C관련 → SPT, ZT 관련 MST, CR, ATC)

- ① 빔필터: 현재의 탐색트리 수준에서 실행가능한 해를 평가할 노드수 나눌 수 있음.
- ② 빔너비: 빔필터에 의해서 해를 평가한 노드 중에서 하위수준으로 탐색을 진행할 노드수

Largest Tail Procedure (LTP) 규칙 → Cmax, Lmax, 최소화 (or Tmax)

도착시간 r_i, 작업시간 p_i, 배달시간 g_i → C_i = r_i + p_i + g_i (t + w_i : r, p 사이 대기)

Next job = argmax_i {g_i} ① 현재시점 C_i = r_i + w_i + p_i 계산, ② D 설정 = max{d_i} + 1

③ 배달시간 g_i = D - d_i로 설정 ④ Lmax = max{C_i - d_i} = max{r_i + w_i + p_i + g_i} - D → Cmax 최소화

분지한계법 → LB 추정. DFS/BFS로 branch. IC 찾기. LB보다 크면 fathomed.

작업의 준비시간만 존재하는 경우: 기계점유시간 = S_ij + p_j : S_ij가 seq. dep. 이라면 점유시간 = 변수

TSP 문제 → Cmax = Σ_{i=1}^{n-1} S_{(i-1), i} + Σ_{i=1}^n p_i = Σ S_ij 최소화 (1 ≤ i ≤ Cmax)

① Closest unscheduled job (CUT) 규칙 (Greedy)

첫 작업 임의로 선택 → 가장 작은 S_ij로 다음작업 선택. (최작업 매개에 대해서 선택후 선택 아)

② Look ahead 규칙 (if m=n, 원순제와 동일) (m: look ahead 수)

최작업이 있을 때, n개를 인접작업 m개까지 부분순제 → 가장 소 S_ij 가 작은 부분조합 선택.

③ k-opt 규칙 (k=2 → 2-opt 규칙): 최실행가능해

④ LP로 BNB Min Σ_i Σ_j S_ij x_ij s.t. Σ_j x_ij = 1 ∀ i, Σ_i x_ij ≤ V_j

u_i - u_j + n x_ij ≤ n - 1 ∀ i, j, i ≠ j or Σ_{i ∈ S} Σ_{j ∈ S^c} x_ij ≤ |S| - 1 ∀ S ⊂ V, S ≠ ∅

x_ij ∈ {0, 1} ∀ i, j, i ≠ j. (LP relax. 결과, 이걸 LB), u_i ≥ 0 ∀ i. Integer.

⑤ DP 단계 k = 고려할 작업수. 상여 J = 각 단계에서 가능한 작업 부분집합.

f_k(i, J) = 단계 k에서 작업 J가 다음에

부속한 J에 속한 작업들을 거쳐

가장 V로 들어오는 최소비용.

= min_{j ∈ J} {S_ij + f_{k-1}(j, J - {i})}

f_k(i, ∅) = 0, k = 0.

	1(v)	2	3	4
1(v)	-	0	0	0
2	0	-	20	80
3	0	30	-	60
4	0	15	10	-

원점 v = 1	
단계 0	
f_0(2, ∅)	f_0(3, ∅)
f_0(4, ∅)	
단계 1	
f_1(2, {3}) = 20 + 0	f_1(2, {4}) = 80 + 0
f_1(3, {2}) = 30 + 0	f_1(3, {4}) = 60 + 0
f_1(4, {2}) = 15 + 0	f_1(4, {3}) = 10 + 0
단계 2	
f_2(2, {3, 4}) = min{20 + 60, 80 + 10} = 80	
f_2(3, {2, 4}) = min{30 + 80, 60 + 15} = 75	
f_2(4, {2, 3}) = min{15 + 20, 10 + 30} = 35	
단계 3	
f_3(1, {2, 3, 4}) = min{0 + 80, 0 + 75, 0 + 35} = 35	
최적 스케줄: (1, 4, 2, 3, 1)	
총 작업 준비시간: 35	

동적작업 도착시간 & 작업준비시간 동시에 존재하는 경우

Apparent Tardiness Cost with Setup (ATCS) 규칙 (1/r_i, S_ij | ΣWT, n_T)

I_i(t|i)는 시간 t에 작업 i가 완료되었을 때 다음 작업 후보 중 작업 j의 우선순위를 결정하는 값으로 다음과 같이 정의한다.

(합성) I_i(t|i) = (W_i / p_i) * exp(-max{0, d_i - p_i - t} / k_1 * p̄) * exp(-S_ij / k_2 * s)

시간 t에서 우선순위가 가장 높은 작업 j가 작업 i 바로 뒤에 위치한다. p̄는 시간 t에 기계 앞에서 기다리는 작업들의 평균 가공시간이고 s는 평균 준비 시간이다.

• 모수 k_1은 ATC 규칙과 마찬가지로 Due date scaling parameter로서 다음과 같이 설정 된다.

k = 4.5 + R for R ≤ 0.5
k = 6 - 2R for R > 0.5

where R = (d_max - d_min) / C_max ≈ due date range factor

• 한편 모수 k_2는 Setup time scaling parameter로서 다음과 같이 설정된다.

k_2 = τ / (s * √η)

where τ = 1 - d̄ / C_max and η = s̄ / p̄

여기서 τ는 Due date tightness, η는 Setup time severity 라고 부른다.

• 주의해야 할 점은 1/r_i | Σ_{i=1}^n w_i T_i 문제와 달리 1/r_i, S_ij | Σ_{i=1}^n w_i T_i 문제에서는 C_max가 상수가 아니므로 미리 알 수 없다. 따라서 n개의 작업이 있을 때 보통 다음 식을 이용해서 C_max를 예측한다.

C_max ≈ Σ_{i=1}^n (p_i + n * s̄)

ERT (1/r_i | C_max) (FFC, Om, Jc | recrc, r_j | C_max)
MST (FFC, Om, Jc | recrc, r_j | T̄)
CR (FFC, Om, Jc | recrc, r_j | T̄)
ATC (FFC, Om, Jc | recrc, r_j | T̄)
MMD (FFC, Om, Jc | recrc, r_j | T̄)
COVERT (FFC, Om, Jc | recrc, r_j | T̄)
LTP (HBT-형태, C_max)

병렬기계 스케줄링 → workload balancing

Pm: 기계 성능이 같은 병렬 환경. Qm: 성능 다른데, 기계마다 속도를 일정하게 정적. Rm: 속도의 X (Pm | prmp | Cmax) 작업의 선택비용 (기계에서 가공중인 작업중간 기능)

Cmax* = max {sum(Pi)/m, max(pi)} → sum(Pi)/m 보다 p가 큰 경우, 작업의 splitting 불가능.
① 한 기계에 Cmax* 이하로 배치 ② 나머지는 다른 기계로 / 선택비용 X, Cmax ≥ max() → LB로 작용.
(Pm | (prec) | Cmax) List Scheduling

정렬된 작업에서 가장 첫 작업을 기계들 중에 가장 빨리 가공을 시작할 수 있는 기계에 배정.
(선행조건 X) 성능: 1 ≤ Cmax/Cmax* ≤ 2 - 1/m ≤ 2, p=2 근사 알고리즘

(선행조건 O) 선행조건이 자연인 목록 생성 후, 가장 작업을 빨리 시작할 수 있는 기계로. m이 작을수록 효과 ↑
성능: Cmax ≤ sum(Pi) + sum(Pi)/m ≤ 2Cmax*, p=2 근사 알고리즘.

(Pm || Cmax) LPT scheduling

가공시간이 큰 순서대로 목록 구성 → 가장 빨리 가공 시작할 수 있는 기계에 배정.
성능: Cmax/Cmax* ≤ 4/3 - 1/3m

- (1) LPT 스케줄링은 List 스케줄링 보다 효과는 크다. 다시 말해서 List 스케줄링은 ρ = 2 근사 알고리즘인 반면에 LPT 스케줄링은 ρ = 4/3 ≈ 1.33 근사 알고리즘이다. Worst case 가 LPT가 더 좋음.
- (2) 기계 수 (m)가 증가함에 따라 불구하고 LPT 스케줄링의 성능은 최적화 방법에 비해서 크게 떨어지지 않는다. List는 online 가능, LPT는 offline만 가능.
- (3) 그러나 스케줄 할 작업 집합을 (또는 작업의 가공시간들) 미리 알 수 없다면 LPT 스케줄링을 사용할 수 없고 이 때는 List 스케줄링이 보다 현실적인 대안이다.

(Pm || Cmax) Multifit algorithm

성능: Cmax/Cmax* ≤ 1.2 + 1/2k (k: FFD 반복 횟수)
사용자가 원하는 Makespan M 설정. → FFD 절차 (LPT 진행하고 M이내 안되면 M을 늘려서 재시도)
실패하면 M 증가 후 반복. LB = max {sum(Pi)/m, max(pi)}. UB = 2LB. 최 M = 1/2(LB + UB) < 작업할 수 있는 UB = M 반복

(Pm || F) nj: 기계 j에 할당된 작업수. p(i,j): i번째 가공되는 작업의 가공시간
F = sum_{i=1}^{nj} p(i,j) / nj → p(i,j)가 감소하면 배율 (SPT) ① nj 개는 최대 하나 차이 나도록
작업수 개수보다 많다면, 각 기계에 배정된 작업수의 차이는 최대 1개. ② SPT로 목록 구성 → List 스케줄

(Pm || sum(WT)) Lagrangian Relaxation

(P) Z = Min cX → (LRλ) Z(λ) = Min cX + λ(Ax - b)
s.t. Ax ≤ b s.t. Dx ≤ e
Dx ≤ e x ≥ 0, int x ≥ 0, int (λ ≥ 0)
→ Z(λ) ≤ Z: LRλ는 P의 LB를 제공 → LB 중 가장 tight: Z0 = maxλ Z(λ)
(D) Z0 = Max w s.t. w ≤ cXk + λ(Axk - b) ∀ k
주어진 Xk에 대해서 λ를 찾는 문제 Xk: 실행 가능한 해집합.

① t=0, λ^0=0 ② λ^t에서 목적함수 Z(λ^t)의 subgrad s^t = Ax^t - b를 구함.
if s^t ≈ 0, λ^t 최적해. ③ λ^{t+1} = max{0, λ^t + γ s^t}, t = t+1 업데이트

이때 γ^t = μ^t * (Z* - Z(λ^t)) / sum_{i=1}^m (b_i - sum_{j=1}^n a_ij x_i^t) → 차이가 크면 큰 step size ↑ (최대는 ↑ ↓)
→ 차이가 크면 큰 step size ↓ 수렴

- (1) Z*는 현재까지 찾은 또는 추정된 원 문제 P의 가장 좋은 해의 목적함수 값
- (2) μ^t는 t가 증가함에 따라 감소하는 모수 값으로 초기 값으로 0 < μ^0 ≤ 2을 만족

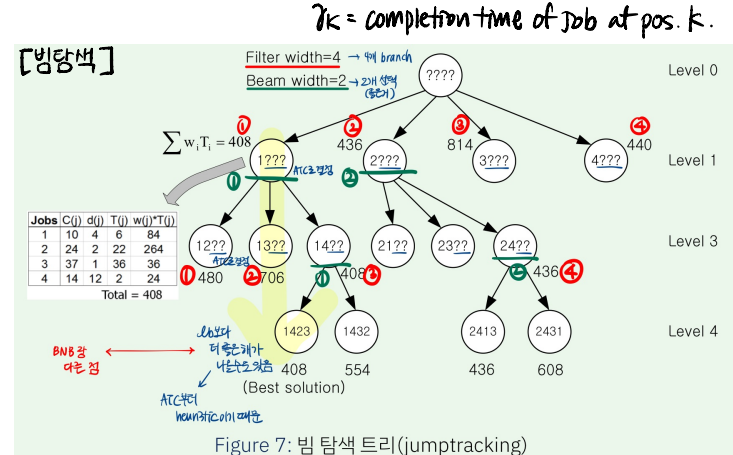
μ^{t+1} = { αμ^t Z0가 T회수 동안 증가하지 않는다면, μ^t 그렇지 않다면 }
0 < α < 1 그리고 T > 1

라그랑지안 완화 기법의 핵심적인 사항

- (1) 원 문제 P에 대한 라그랑지안 완화 문제 LRλ는 하한 경계를 제공한다.
- (2) 라그랑지안 완화 문제 LRλ는 λ가 고정되어 있을 때 최적의 해 x를 찾는 문제이다.
- (3) 강한 하한 경계가 분지 한계법의 효율성을 증가시키기 때문에 LRλ를 최대화하는 Z0 문제를 만들었다.
- (4) Z0 = maxλ Z(λ) 문제에서의 결정변수는 λ이다. 그리고 해 공간은 λ에 대해서 Concave 형태의 연속 함수이다. 하지만 미분은 가능하지 않다. 따라서 Subgradient 탐색방법을 이용해 최적의 λ를 찾는다. 최적의 λ를 목적함수에 대입한 결과 값이 Z0이며 이 값이 원 문제의 강한 하한 경계 값이다.

LP로 BnB (C1 | r1, Sij | Total or Average) - 4가지

- Completion time: yjk = 1 (j is processed before k)
 - Time index: xjt = 1 (j starts at t)
 - Linear ordering: δjk = 1 (j precedes k)
 - Assignment / positional date ujk = 1 (j is assigned to k)
- rk = completion time of job at pos. k.



[기계환경]

- 단일 (1): njobs → □
- 병렬: 동일병렬(Pm) Pm1 = Pm2 = P; 동일(Qm) 기계마다 각각도 Rm 존재 Pm = P x Rm; 독립(Rm) Rm 정의 안됨
- 흐름작업장 (Fm) njobs → □ ... □
- 유연동작업장 (FFc) njobs → (□) → (□) → ... (□) m stage
- 개방작업장 (Om) □ ... □
- 개별작업장 (Jc) (□) → (□) → ... (□)

순서지: Fm, FFc, stage 구성 O: FFc, Jc
[Forward DP]

g(S, j) = (1) S ⊆ {2, 3, 4}, j ∈ S Stage 1 (|S|=1)
점화식: g({2}, 2) = s12 = 0, g({3}, 3) = s13 = 0, g({4}, 4) = s14 = 0
Stage 2 (|S|=2)
(S={2,3})
g({2,3}, 2) = g({3}, 3) + s3,2 = 0 + 30 = 30
g({2,3}, 3) = g({2}, 2) + s2,3 = 0 + 20 = 20
(S={2,4})
g({2,4}, 2) = g({4}, 4) + s4,2 = 0 + 15 = 15
g({2,4}, 4) = g({2}, 2) + s2,4 = 0 + 80 = 80
(S={3,4})
g({3,4}, 3) = g({4}, 4) + s4,3 = 0 + 10 = 10
g({3,4}, 4) = g({3}, 3) + s3,4 = 0 + 60 = 60
Stage 3 (|S|=3, S={2,3,4})
마지막이 2인 경우
g({2,3,4}, 2) = min{g({3,4}, 3) + s3,2, g({3,4}, 4) + s4,2} = min{30 + 30, 80 + 15} = min{60, 95} = 60
→ predecessor는 3
마지막이 3인 경우
g({2,3,4}, 3) = min{g({2,4}, 2) + s2,3, g({2,4}, 4) + s4,3} = min{20 + 20, 80 + 10} = min{40, 90} = 40
→ predecessor는 2
마지막이 4인 경우
g({2,3,4}, 4) = min{g({2,3}, 2) + s2,4, g({2,3}, 3) + s3,4} = min{30 + 80, 20 + 60} = min{110, 80} = 80
→ predecessor는 3
복귀비용 sji = 0 이므로 최종 최소는 min{40, 35, 80} = 35
즉 마지막은 3.
경로를 거꾸로 따라가면:
• 마지막 3의 predecessor는 2 (위에서 선택)
• 상태 {2,4}에서 마지막 2의 predecessor는 4
• 시작은 1